Memoria explicativa - Práctica 7

Consultas SQL desde Java mediante JDBC. Los ejercicios 1-5 trabajan con PostgreSQL y la base **escuela**; los ejercicios 6-10, con MySQL y la base **practica**. Los bloques *try-with-resources* cierran las conexiones y resultados al terminar.

Ejercicio 1 - Alumnos de Madrid

Consulta PostgreSQL para buscar alumnos cuya población es Madrid. Muestra DNI, nombre y teléfono, ordenados por nombre.

```
package Practica7;

import java.sql.*;

public class Ejercicio_1_BusquedaMadrid {
    // Datos de conexión de la base de referencia del curso (docker-compose)
    static final String URL  = "jdbc:postgresql://localhost:5432/escuela";
    static final String USER = "postgres";
    static final String PASS = "oretania";
    static String sql;
    public static void main(String[] args) {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS)) {

            sql = "SELECT dni, apenom, telef FROM alumnos where pobla = 'Madrid' ORDER BY apenom";
            try (Statement st = con.createStatement();
                 ResultSet rs = st.executeQuery(sql)) {
                System.out.println("ALUMNOS:");
                while (rs.next()) {
                    System.out.printf("  %-10s %-25s %s%n",
                            rs.getString("dni"),
                            rs.getString("apenom"),
                            rs.getString("telef"));
                }
            }
        } catch (SQLException e) {
            System.err.println("Error SQL [" + e.getSQLState() + "]: " + e.getMessage());
        }
    }
}
```

Ejercicio 2 - Notas por alumno y asignatura

Relaciona alumnos, notas y asignaturas para mostrar el nombre del alumno, la asignatura y la nota. Ordena por alumno y por nota descendente.

```
package Practica7;

import java.sql.*;

public class Ejercicio_2_VIsualizarAlumno {
    static final String URL  = "jdbc:postgresql://localhost:5432/escuela";
    static final String USER = "postgres";
    static final String PASS = "oretania";
    static String sql;
    public static void main(String[] args) {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS)) {

            sql = "SELECT apenom, nombre as asignatura, nota FROM alumnos JOIN notas ON notas.dni = alumnos.dni JOIN asignaturas " +
                    "ON asignaturas.cod = notas.cod ORDER BY alumnos.apenom ASC, alumnos.dni ASC, notas.nota DESC";
            try (Statement st = con.createStatement();
                 ResultSet rs = st.executeQuery(sql)) {
                System.out.println("ALUMNOS:");
                while (rs.next()) {
                    System.out.printf("  %-10s %-25s %s%n",
                            rs.getString("apenom"),
                            rs.getString("asignatura"),
                            rs.getString("nota"));
                }
            }
        } catch (SQLException e) {
            System.err.println("Error SQL [" + e.getSQLState() + "]: " + e.getMessage());
        }
    }
}
```

Ejercicio 3 - Número de notas y nota media

Agrupa las notas por alumno, cuenta sus asignaturas con nota y calcula la media con dos decimales. Ordena de mayor a menor media.

```
package Practica7;

import java.sql.*;

public class Ejercicio_3_NotaMedia {
    static final String URL = "jdbc:postgresql://localhost:5432/escuela";
    static final String USER = "postgres";
    static final String PASS = "oretania";
    static String sql;

    public static void main(String[] args) {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS)){
            sql = "select apenom, count(notas) as numAsig, round(avg(nota), 2) as media from alumnos left join notas on " +
                    "alumnos.dni = notas.dni group by alumnos.dni, alumnos.apenom order by media desc nulls last";
            try (Statement st = con.createStatement();
                 ResultSet rs = st.executeQuery(sql)) {
                System.out.println("Nombre y Media:");
                while (rs.next()) {
                    System.out.printf("%-30s | %2d asignaturas | %.2f%n",
                            rs.getString("apenom"),
                            rs.getInt("numAsig"),
                            rs.getDouble("media"));
                }
            }
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
}
```

Ejercicio 4 - Nota más alta

Busca la nota máxima de la base de datos y muestra el alumno, la asignatura y la nota. Si hay empate, aparecen todas las filas máximas.

```
package Practica7;

import java.sql.*;

public class Ejercicio_4_NotaMasAlta {
    static final String URL = "jdbc:postgresql://localhost:5432/escuela";
    static final String USER = "postgres";
    static final String PASS = "oretania";
    static String sql;

    public static void main(String[] args) {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS)){
            sql = "select apenom, nombre, nota from alumnos join notas on alumnos.dni = notas.dni join asignaturas " +
                    "on notas.cod = asignaturas.cod  where nota = (select max(nota) from notas) order by apenom, alumnos.dni";
            try (Statement st = con.createStatement();
                 ResultSet rs = st.executeQuery(sql)){
                System.out.println("Alumnos con nota más alta:");
                while (rs.next()) {
                    System.out.printf("%-30s | %-30s | %d%n",
                            rs.getString("apenom"),
                            rs.getString("nombre"),
                            rs.getInt("nota"));
                }
            }
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
}
```

Ejercicio 5 - Asignaturas por encima de la media global

Calcula la media de cada asignatura y conserva aquellas que superan la media de todas las notas.

```
package Practica7;

import java.sql.*;

public class Ejercicio_5_MayorMedia {
    static final String URL = "jdbc:postgresql://localhost:5432/escuela";
    static final String USER = "postgres";
    static final String PASS = "oretania";
    static String sql;

    public static void main(String[] args) {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS)){
            sql = "select nombre, round(avg(nota), 2) as nota from asignaturas join notas on asignaturas.cod = notas.cod group by nombre " +
                    "having avg(nota) > (select avg(nota) from notas) order by nota desc";
            try (Statement st = con.createStatement();
                 ResultSet rs = st.executeQuery(sql)){
                System.out.println("Asginatura y nota");
                while (rs.next()) {
                    System.out.printf("%-30s",
                            rs.getString("nombre"));
                }
            }
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
}
```

Nota: En el archivo actual, la consulta obtiene también la media de cada asignatura, pero la salida Java imprime solo el nombre.

Ejercicio 6 - Empleados de un departamento

Consulta los empleados del departamento número 1 y muestra nombre, apellido, oficio y salario.

```
package Practica7;

import java.sql.*;

public class Ejercicio_6_Practica {
    static final String URL = "jdbc:mysql://localhost:3306/practica";
    static final String USER = "practica";
    static final String PASS = "practica";
    static String sql;

    public static void main(String[] args) {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS)){
            sql = "select nombre, apellido, oficio, salario from empleados where dept_no = 1 order by apellido";
            try (Statement st = con.createStatement();
                 ResultSet rs = st.executeQuery(sql)){
                System.out.println("Empleados:");
                while (rs.next()) {
                    System.out.println(rs.getString("nombre") + " " + rs.getString("apellido") +
                            " " + rs.getString("oficio") + " " + rs.getDouble("salario"));
                }
            }
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
}
```

Ejercicio 7 - Salario máximo

Busca a los empleados con el salario máximo y muestra su apellido, salario y departamento.

```
package Practica7;

import java.sql.*;

public class Ejercicio_7_MaximoSalario {
    static final String URL = "jdbc:mysql://localhost:3306/practica";
    static final String USER = "practica";
    static final String PASS = "practica";
    static String sql;

    public static void main(String[] args) {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS)){
            sql = "select apellido, salario, dnombre from empleados join departamentos on empleados.dept_no = departamentos.dept_no where salario
                = (select max(salario) from empleados) order by apellido";
            try (Statement st = con.createStatement();
                 ResultSet rs = st.executeQuery(sql)){
                while (rs.next()) {
                    System.out.println(rs.getString("apellido") + " " + rs.getDouble("salario") + " " + rs.getString("dnombre"));
                }
            }
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
}
```

Ejercicio 8 - Resumen por departamento

Para cada departamento calcula cuántos empleados tiene y su salario medio. El LEFT JOIN permite incluir departamentos sin empleados.

```
package Practica7;

import java.sql.*;

public class Ejercicio_8_VisualizarDept {
    static final String URL = "jdbc:mysql://localhost:3306/practica";
    static final String USER = "practica";
    static final String PASS = "practica";
    static String sql;

    public static void main(String[] args) {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS)){
            sql = "select dnombre, count(emp_no) as numero_empleados, coalesce(round(avg(salario), 2), 0.00) as salario_medio from departamentos
                left join empleados on departamentos.dept_no = empleados.dept_no group by departamentos.dept_no, departamentos.dnombre order by
                dnombre";
            try (Statement st = con.createStatement(); ResultSet rs = st.executeQuery(sql)){
                while (rs.next()) {
                    System.out.println(rs.getString("dnombre") + " " + rs.getInt("numero_empleados") + " " + rs.getDouble("salario_medio"));
                }
            }
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
}
```

Ejercicio 9 - Contrataciones por fecha

Muestra apellido, fecha de alta y departamento para empleados contratados dentro del intervalo indicado en la consulta.

```
package Practica7;

import java.sql.*;

public class Ejercicio_9_MarzoJunio {
    static final String URL = "jdbc:mysql://localhost:3306/practica";
    static final String USER = "practica";
    static final String PASS = "practica";
    static String sql;

    public static void main(String[] args) {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS)){
            sql = "select apellido, fecha_alt, dnombre from empleados join departamentos on empleados.dept_no = departamentos.dept_no where
                fecha_alt between '2023-05-01' and '2023-06-30'";
            try (Statement st = con.createStatement(); ResultSet rs = st.executeQuery(sql)){
                while (rs.next()) {
                    System.out.println(rs.getString("apellido") + " " + rs.getString("fecha_alt") + " " + rs.getString("dnombre"));
                }
            }
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
}
```

Nota: El nombre del archivo menciona marzo-junio, pero la consulta actual filtra del 1 de mayo al 30 de junio de 2023.

Ejercicio 10 - Salarios superiores a la media

Muestra apellido, oficio, salario y departamento de los empleados cuyo salario supera la media de todos los empleados.

```
package Practica7;

import java.sql.*;

public class Ejercicio_10_MayorSalario {
    static final String URL = "jdbc:mysql://localhost:3306/practica";
    static final String USER = "practica";
    static final String PASS = "practica";
    static String sql;

    public static void main(String[] args) {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS)){
            sql = "select apellido, oficio, salario, dnombre from empleados join departamentos on empleados.dept_no = departamentos.dept_no where
                salario > (select avg(salario) from empleados)";
            try (Statement st = con.createStatement(); ResultSet rs = st.executeQuery(sql)){
                while (rs.next()) {
                    System.out.println(rs.getString("apellido") + " " + rs.getString("oficio") + " " + rs.getDouble("salario") + " " +
                        rs.getString("dnombre"));
                }
            }
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
}
```